Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title: Seamless Appointment Booking for Health**
- **Team Members: Mamatha & Vijay: Front-end , Jaswanth:Back-end, Sowmya&Jayasree: Database,Documentation.**

2. Project Overview

- **Purpose**: This project aims to provide a user-friendly platform for patients to book appointments with healthcare providers online. It reduces wait times, avoids manual errors, and ensures smooth scheduling and reminders for both doctors and patients.
- **Features:**
- **Patient and doctor registration/login**
- **Search doctors by specialty/location**
- **Real-time appointment slot availability**
- **Secure booking and rescheduling**
- **Email/SMS appointment confirmations**
- **Admin panel to manage users and schedules**

3. Architecture

- **Frontend: Developed using React.js with components for login, registration, booking, calendar view, and admin dashboard. Styled with Tailwind CSS.**
- **Backend: Built using Node.js and Express.js. RESTful APIs handle user management, booking operations, and role-based access control.**
- Database:
- MongoDB stores user data, appointment details, doctor availability, and system logs. Mongoose is used for schema modeling

4. Setup Instructions

- **Prerequisites: Node.js, MongoDB, npm or yarn**
- **Installation: 1. Clone the repo:git clone https://github.com/your-repo/health-appointment-booking.git**
  2. Install dependencies:
  cd client
  npm install
  cd ../server
  npm install
  3. Set environment variables in a .env file (e.g., DB URI, JWT secret).

5. Folder Structure

- **Client: App.js,index.js.**
- **Server:** server.js,config.js.

6. Running the Application

  - **Frontend:** `cd client,npm start.`
  - **Backend:** `cd server,npm start.`

7. API Documentation

Documented using Swagger/Postman:

POST /api/auth/register – Register new users

POST /api/auth/login – User login

GET /api/doctors – List all doctors

GET /api/doctors/:id – View doctor details

POST /api/appointments – Book appointment

GET /api/appointments/user/:id – Get user appointments

DELETE /api/appointments/:id – Cancel appointment

8. Authentication

- Uses JWT (JSON Web Tokens) for user authentication.
- Role-based authorization for patients, doctors, and admins.
- Tokens are stored in HTTP-only cookies or local storage.
- Middleware protects private routes.

9. User Interface

The application has a clean and intuitive interface using React.js and Tailwind CSS.
Navigation is organized for three types of users: Patients, Doctors, and Admins.
Features include:
Appointment booking calendar
Doctor profile cards
User dashboard with upcoming appointments
Admin panel for managing users and appointments
Screenshots / GIFs (to be added)

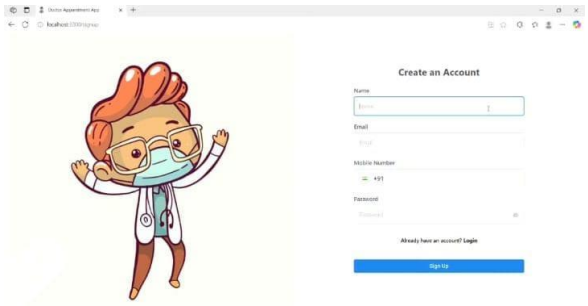
Provide screenshots or GIFs showcasing different UI features.

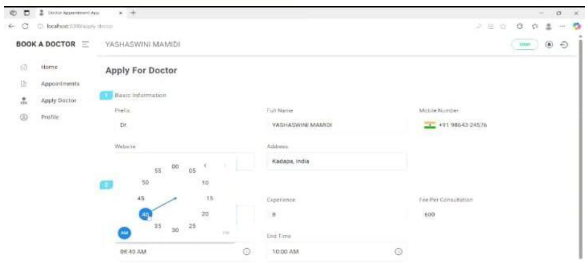
10. Testing

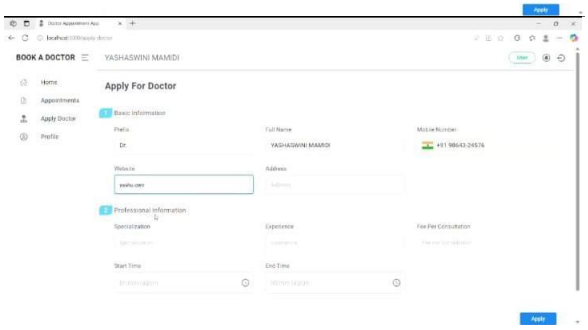
- Unit testing for backend API endpoints.
- Integration testing for booking flow.
- Manual UI testing for various user roles.
- Tools Used:
- Jest and Supertest for backend API testing
- React Testing Library for frontend component testing
- Postman for manual API testing
- MongoDB Compass for database validation

11. Screenshots or Demo

- Screenshots: Booking Interface







. Demo Video Link :
https://drive.google.com/file/d/123RKsHr8mUVLdMTqJJmwL6TYnBkmAnD-/view?usp=drivesdk

12. Known Issues

- Appointment time slots may overlap if multiple users book simultaneously (requires queueing or atomic DB update).
- SMS notification service may not work in all regions due to third-party API restrictions.
- Limited role management features for Admin (future enhancement needed)..

13. Future Enhancements

- Integrate video consultation feature using WebRTC or Zoom API.
- Add calendar sync with Google Calendar or Outlook.
- Implement reminder notifications via SMS/email before appointment time.
- Add payment gateway integration (Razorpay, Stripe) for paid bookings.
- Improve AI-based doctor recommendation based on symptoms.